Software Architecture and Architectural Styles

Lecture 14

Agenda

- Software Design Principles
- Architectural Design vs. Detailed Design
- Software Architecture
- Architectural Design Decisions
- Architectural Views
- Architectural Styles
- Architecture Description Language (ADL)

Software Design Principles

¿Qué es diseño de software?

- Se puede referir a dos cosas: a un proceso o al resultado de este.
- “Diseño de software es un **proceso** que define la **arquitectura, componentes, interfaces** y otras características de un sistema o un componente”.
  - Software Engineering Body of Knowledge (SWEBOK)
- “Un diseño de software es una **descripción de la estructura del software** a ser implementado, **los modelos de datos** y estructuras usadas por el sistema, las **interfaces entre los componentes** del sistema y, algunas veces, los **algoritmos** utilizados”.
  - Ian Sommerville

Components and Connectors

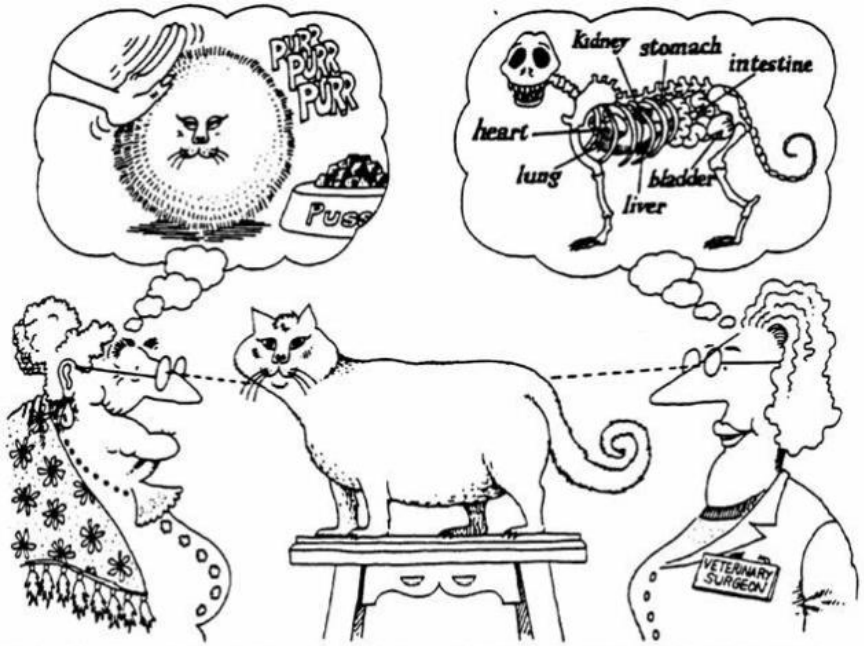
- A software component is an architectural entity that
  - encapsulates a subset of the system's functionality and/or data
  - restricts access to that subset via an explicitly defined interface
  - has explicitly defined dependencies on its required execution context
- A software connector is an architectural building block tasked with effecting and regulating interactions among components

Principios de diseño de software

- Los principios de diseño de software son nociones **fundamentales** que proveen las bases para **diferentes enfoques** de diseño de software. Son **siete**:
  - abstracción;
  - cohesión y acoplamiento;
  - descomposición y modularización;
  - encapsulamiento/ocultamiento de información;
  - separación de interfaz e implementación;
  - suficiencia, completitud y primitivismo;
  - y separación de preocupaciones.

Principios de diseño de software

1. **Abstracción** se refiere a la vista de un objeto enfocándose en la información relevante a un propósito en particular e ignorando el resto de la información.



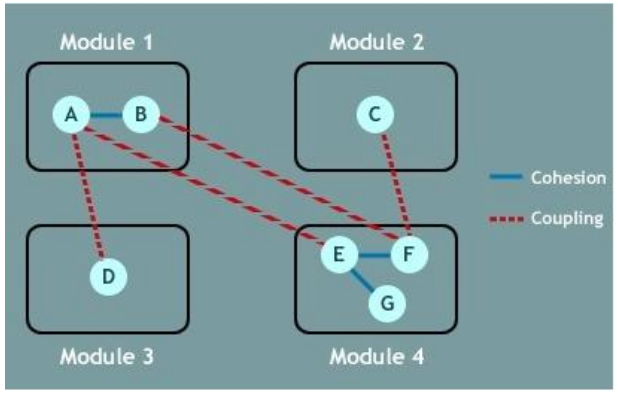
Abstraction focuses upon the essential characteristics of some object, relative to the perspective of the viewer.

Cortesía: Hayim Makabee, The Cat as a Metaphor in Object-Oriented Software Development

Principios de diseño de software

2. **Acoplamiento** se define como el **grado de interdependencia o interconexión entre los módulos** de un software, mientras que **cohesión** se define como el **grado de relación entre las responsabilidades asignadas a un módulo**.
   - Si hay alta interdependencia, entonces los cambios en un módulo tendrán efectos significativos sobre el comportamiento de los demás.
   - Si un módulo es responsable de un número de funciones no relacionadas, entonces las funcionalidades no han sido bien distribuidas entre los módulos.

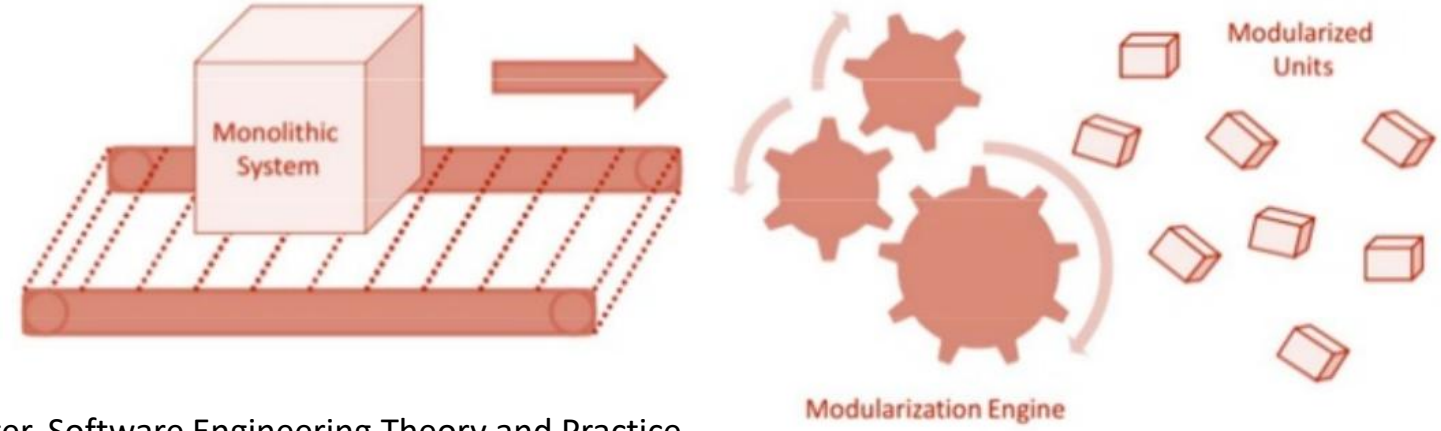
Cortesía: Masudur Rahman, Recommendation of Move Method Refactorings Using Coupling, Cohesion and Contextual Similarity



   - Ideal: **alta cohesión y bajo acoplamiento**.

Principios de diseño de software

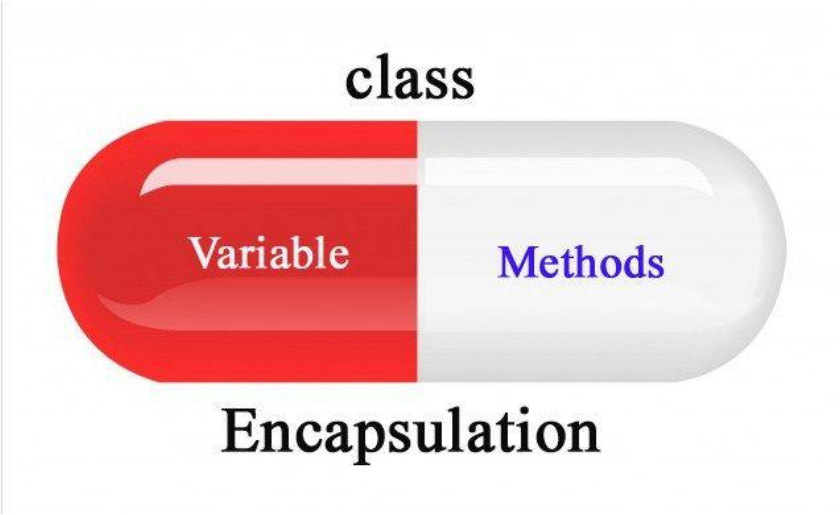
3. **Descomposición y modularización** significa que un software es dividido en un número de componentes más pequeños con interfaces bien definidas que describen las interacciones entre los componentes.
   - Usualmente, el objetivo es ubicar diferentes funcionalidades y responsabilidades en diferentes componentes.



Cortesía: Pfleeger, Software Engineering Theory and Practice

Principios de diseño de software

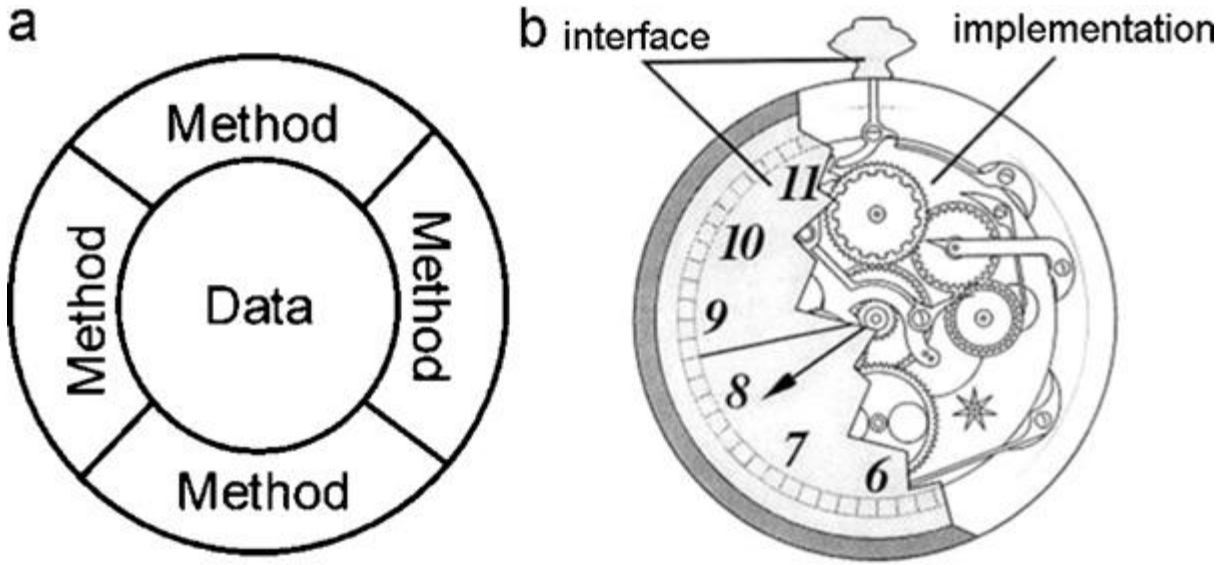
4. **Encapsulamiento y ocultamiento** de información significa agrupar y empaquetar los detalles internos de una abstracción y hacer esos detalles inaccesibles a entidades externas.



Cortesía: Sathasivam Karmehavannan, Encapsulation in Java programming language - Codeforcoding

Principios de diseño de software

5. **Separación de interfaz e implementación** implica definir un componente especificando una interfaz pública (conocida para los clientes) que está separada de los detalles de cómo el componente está implementado.



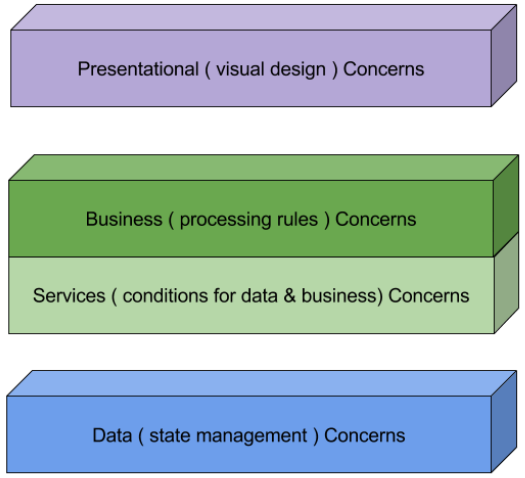
Cortesía: Li An, Modeling Human Decisions in Coupled Human and Natural Systems: Review of Agent-Based Models

Principios de diseño de software

6. **Suficiencia, completitud y primitivismo**: Alcanzar suficiencia y completitud significa asegurar que un componente de software **captura todas las características** importantes para alcanzar su objetivo, no más y no menos. Primitivismo significa que el diseño debería estar basado en patrones que sean de fácil implementación.

Principios de diseño de software

7. **Separación de preocupaciones**: Una preocupación (concern) es un área de interés con respecto al diseño de un software. La arquitectura de un software debe tener una o varias vistas de preocupaciones que permitan a los interesados concentrarse en pocas cosas a la vez a fin de reducir la complejidad.

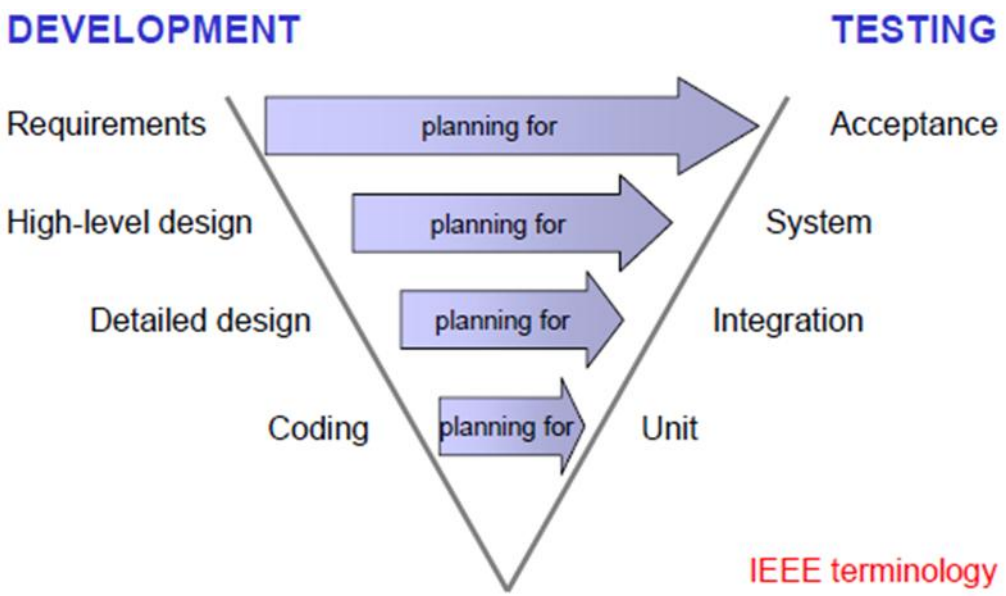


Cortesía: Willie Streeter, Practical Web Development and Architecture

Architectural Design vs. Detailed Design

Diseño de software

- Dos niveles de diseño de software:
  1. Diseño de alto nivel o arquitectural
  2. Diseño detallado



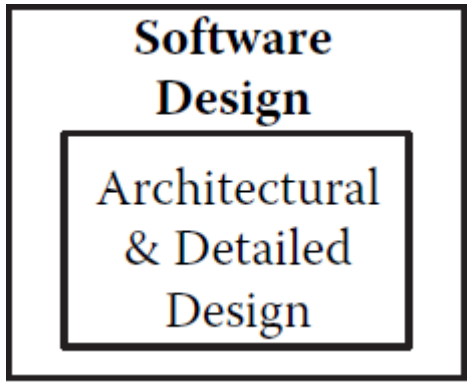
Diseño de alto nivel o arquitectural

- La palabra **arquitectura** es a menudo utilizada en el contexto de algo a **alto nivel**, es decir, está separado de detalles a bajo nivel.
- Es un enfoque de **macro diseño** para crear modelos que ilustren aspectos de calidad y funciones del sistema de software.
- ¿Qué se decide en el diseño arquitectural de un software?
  - **Plataforma** tecnológica
  - **Despliegue** físico del sistema de software, incluyendo subsistemas ubicados en diferentes lugares.
  - Selección de los principales **componentes estructurales**
  - La forma en la que el sistema de software, como un todo, se va a **comunicar** (ej.: protocolos de comunicación)
  - Problemas de **concurrencia**
  - Se evalúa **requerimientos** no funcionales (ej.: desempeño, seguridad, robustez, escalabilidad)
  - Entre otros

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Diseño detallado

- Se **refina** el diseño arquitectural a un punto en el que el **diseño** está lo suficientemente **completo** como para **empezar** la **construcción** del software.
- ¿Qué se decide en el diseño detallado de un software?
  - Se refina componentes en **clases**
  - Se implementan **interfaces**
  - Se especifica las **relaciones** entre clases
  - Se identifica y aplica **patrones** de diseño
  - Se diseña **componentes** y sus interfaces



Software Architecture

Software Architecture

- Architectural design is concerned with understanding how a software system should be organized and designing the overall structure of that system.
- The output of the architectural design process is an architectural model that describes how the system is organized as a set of communicating components.
- "A software system's architecture is the set of principal design decisions made about the system" – Taylor et al.
- "A reference architecture is the set of principal design decisions that are simultaneously applicable to multiple related systems, typically within an application domain, with explicitly defined points of variation" – Taylor et al.

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

A Software Architecture Example

- Each box in the diagram represents a component.
  - Boxes within boxes indicate that the component has been decomposed to subcomponents.
  - Arrows mean that data and or control signals are passed from component to component in the direction of the arrows.

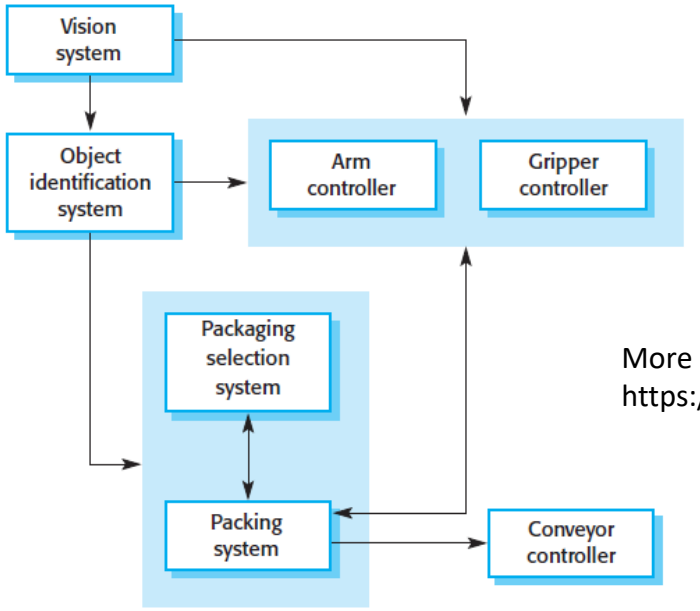


Figure 6.1 The architecture of a packing robot control system

More examples in
https://handbookofsoftwarearchitecture.com

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Why is Software Architecture Important?

- Software architecture is important because it affects the performance, robustness, distributability, and maintainability of a system.
- Individual components implement the functional system requirements, but the dominant influence on the non-functional system characteristics is the system's architecture.

Software Architecture

- Designing and documenting software architecture has three advantages:
  - Stakeholder communication The architecture is a high-level presentation of the system that may be used as a focus for discussion by a range of different stakeholders.
  - System analysis Making the system architecture explicit at an early stage in the system development requires some analysis. Architectural design decisions have a profound effect on whether or not the system can meet critical requirements such as performance, reliability, and maintainability.
  - Large-scale reuse An architectural model is a compact, manageable description of how a system is organized and how the components interoperate. The system architecture is often the same for systems with similar requirements and so can support large-scale software reuse.

Architectural Design Decisions

Architectural Design Decisions

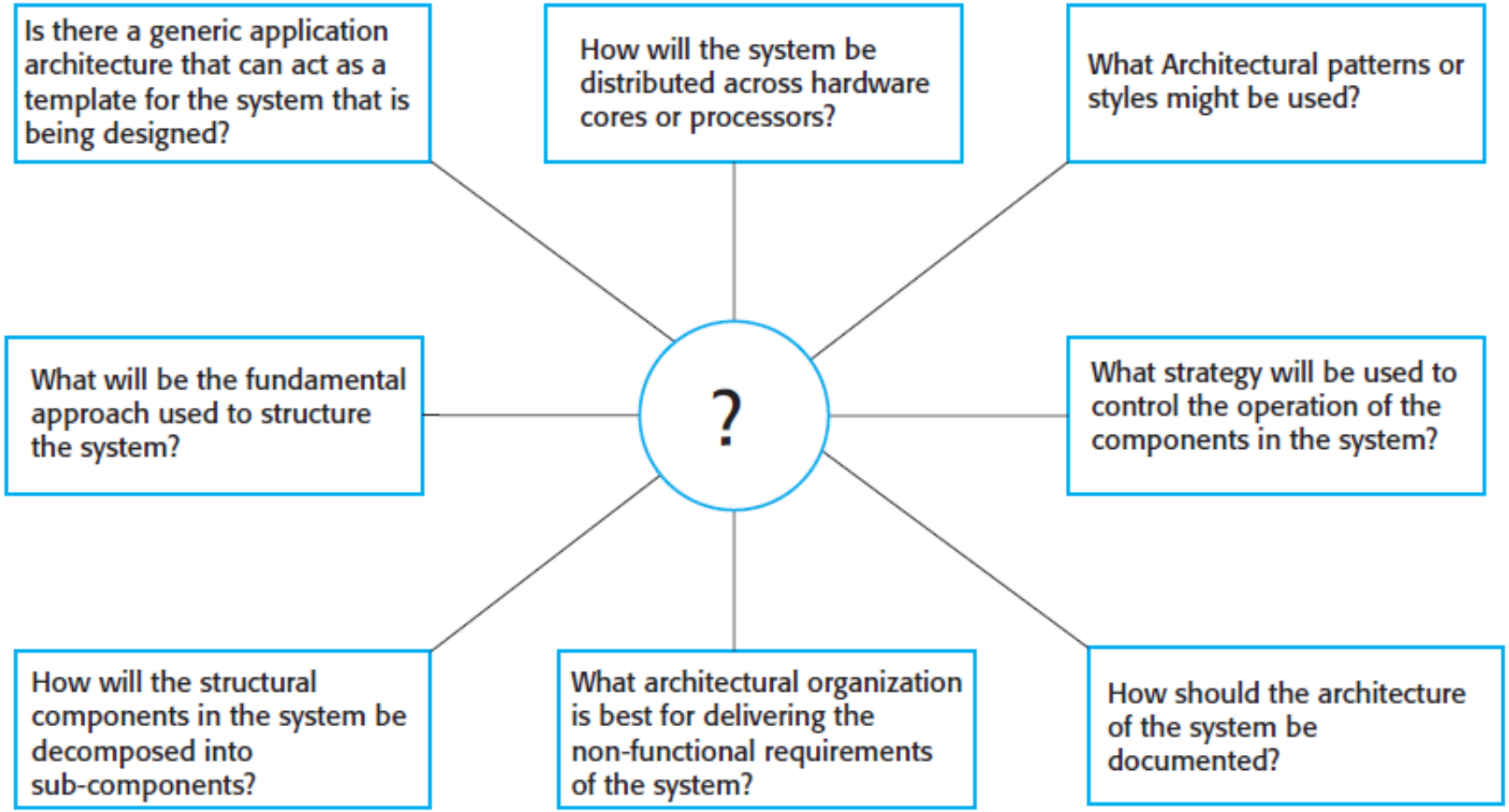


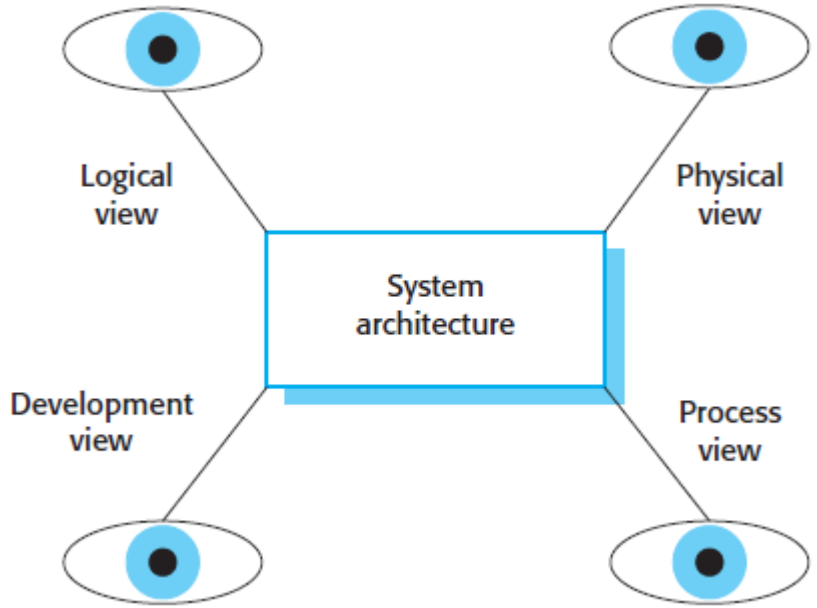
Figure 6.2 Architectural design decisions

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Architectural Views

Architectural Views

- It is impossible to represent all relevant information about a system's architecture in a single diagram, as a graphical model can only show one view or perspective of the system.



Architectural Views

1. A logical view, which shows the key abstractions in the system as objects or object classes. It should be possible to relate the system requirements to entities in this logical view.
2. A process view, which shows how, at runtime, the system is composed of interacting processes. This view is useful for making judgments about non-functional system characteristics such as performance and availability.

Architectural Views

3. A development view, which shows how the software is decomposed for development; that is, it shows the breakdown of the software into components that are implemented by a single developer or development team. This view is useful for software managers and programmers.
4. A physical view, which shows the system hardware and how software components are distributed across the processors in the system. This view is useful for systems engineers planning a system deployment.

Architectural Styles

Architectural Styles

- An architectural style is a named collection of architectural design decisions that
  - are applicable in a given development context
  - constrain architectural design decisions that are specific to a particular system within that context
  - elicit beneficial qualities in each resulting system

Architectural Styles

- The architecture of a software system may be based on a particular architectural pattern or style (these terms have come to mean the same thing).
- However, strictly speaking…
  - Architectural styles define the components and connectors ('what?')
    - ❖Less domain specific
  - Architectural patterns define the implementation strategies of those components and connectors ('how?')
    - ❖More domain specific
- Good architecture makes use of design patterns (on a finer granular level)

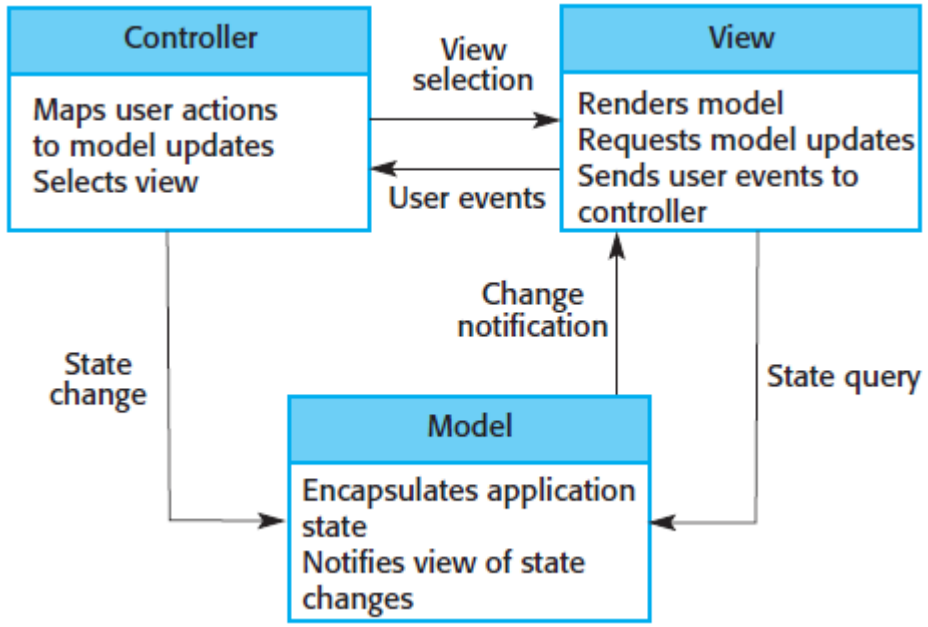
Architectural Styles

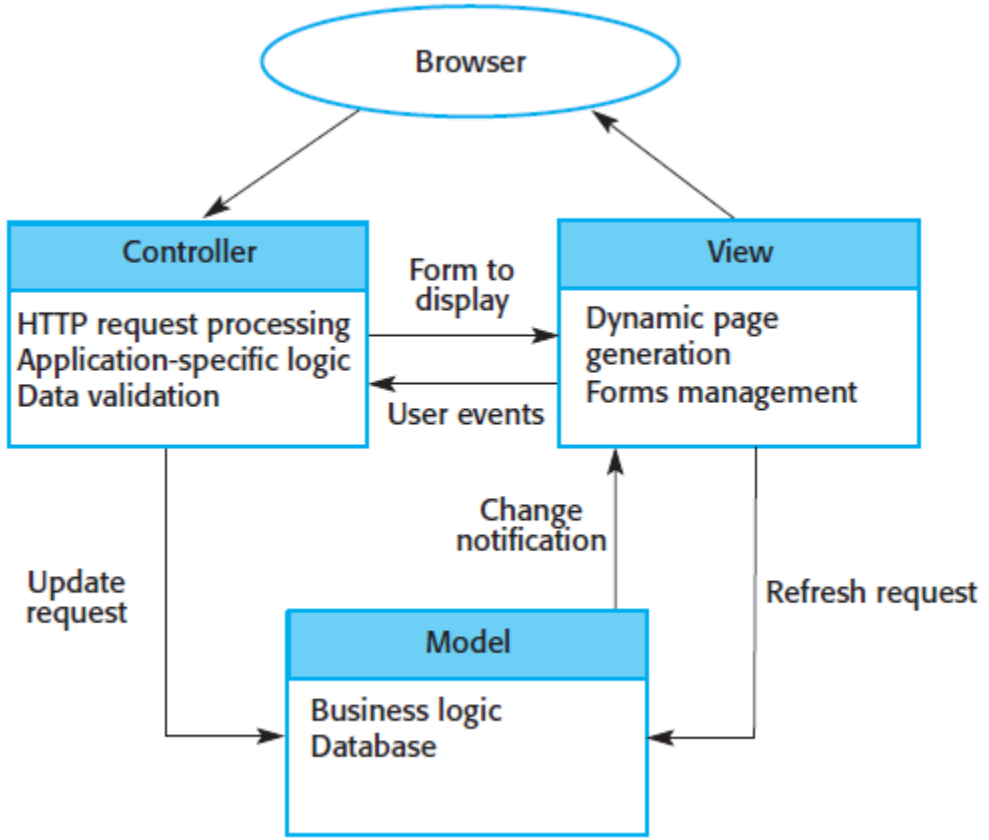
- An architectural style is a description of a system organization, such as a client–server organization or a layered architecture.
- Architectural styles capture the essence of an architecture that has been used in different software systems.
- You should be aware of common styles, where they can be used, and their strengths and weaknesses when making decisions about the architecture.
- Let us introduce a selection of architectural patterns that are commonly used…

The Model-View-Controller (MVC) pattern

Name	MVC (Model-View-Controller)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model. See Figure 6.5.
Example	Figure 6.6 shows the architecture of a web-based application system organized using the MVC pattern.
When used	Used when there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.
Advantages	Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways, with changes made in one representation shown in all of them.
Disadvantages	May involve additional code and code complexity when the data model and interactions are simple.

The Model-View-Controller (MVC) pattern





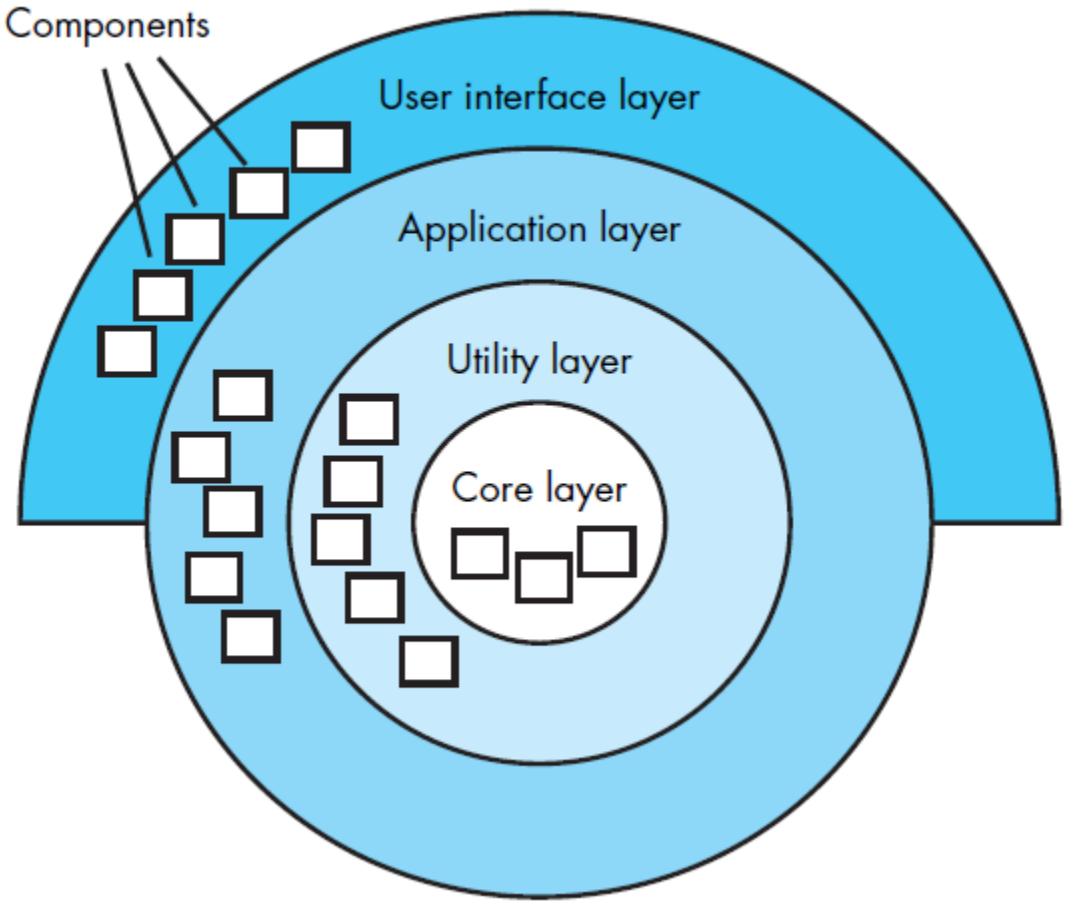
Layered Architecture

Name	Layered architecture
Description	Organizes the system into layers, with related functionality associated with each layer. A layer provides services to the layer above it, so the lowest level layers represent core services that are likely to be used throughout the system. See Figure 6.8.
Example	A layered model of a digital learning system to support learning of all subjects in schools (Figure 6.9).
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multilevel security.
Advantages	Allows replacement of entire layers as long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult, and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

Layered Architecture

FIGURE 13.4

Layered architecture



Layered Architecture

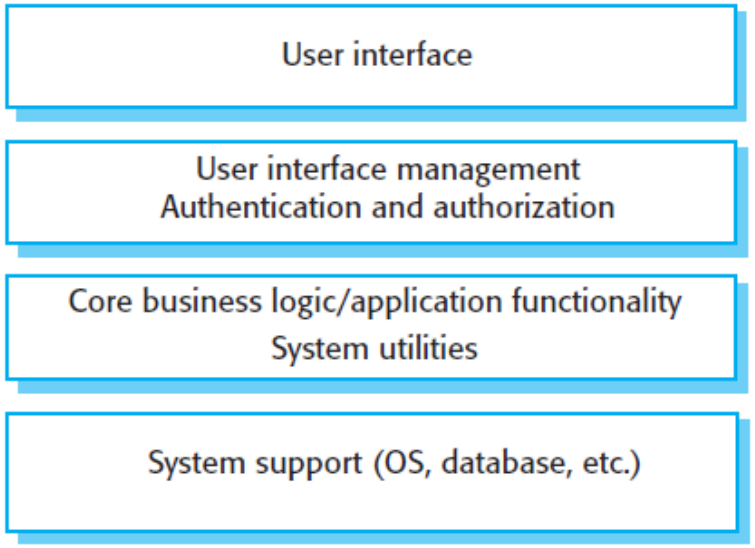


Figure 6.8 A generic layered architecture

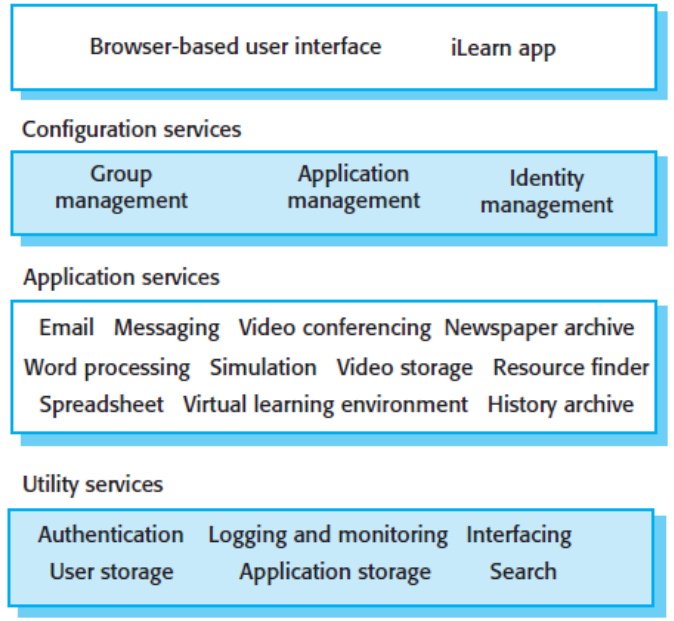
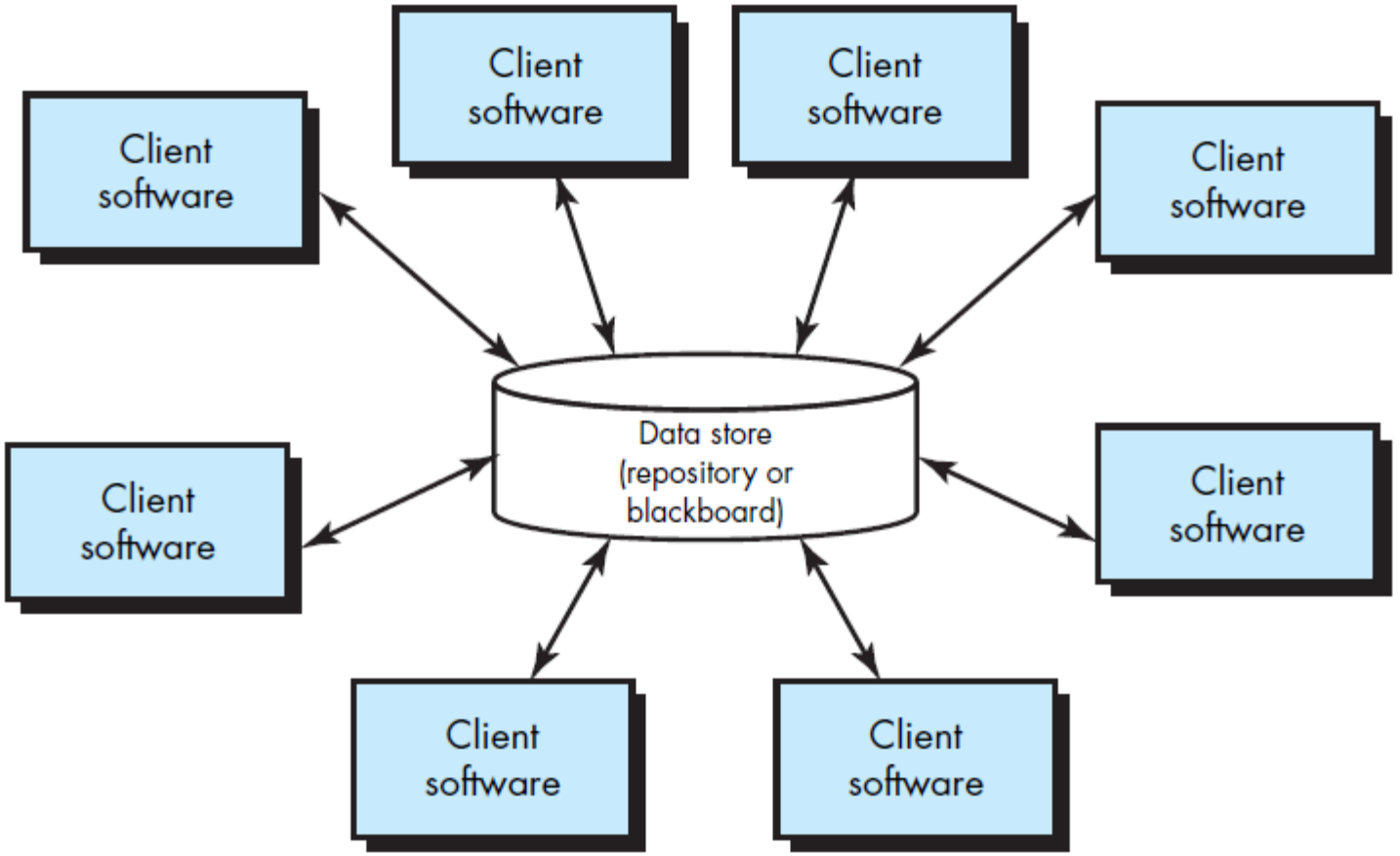


Figure 6.9 The architecture of the iLearn system

Repository Architecture

Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	Figure 6.11 is an example of an IDE where the components use a repository of system design information. Each software tool generates information, which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent; they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

Repository Architecture



Repository Architecture

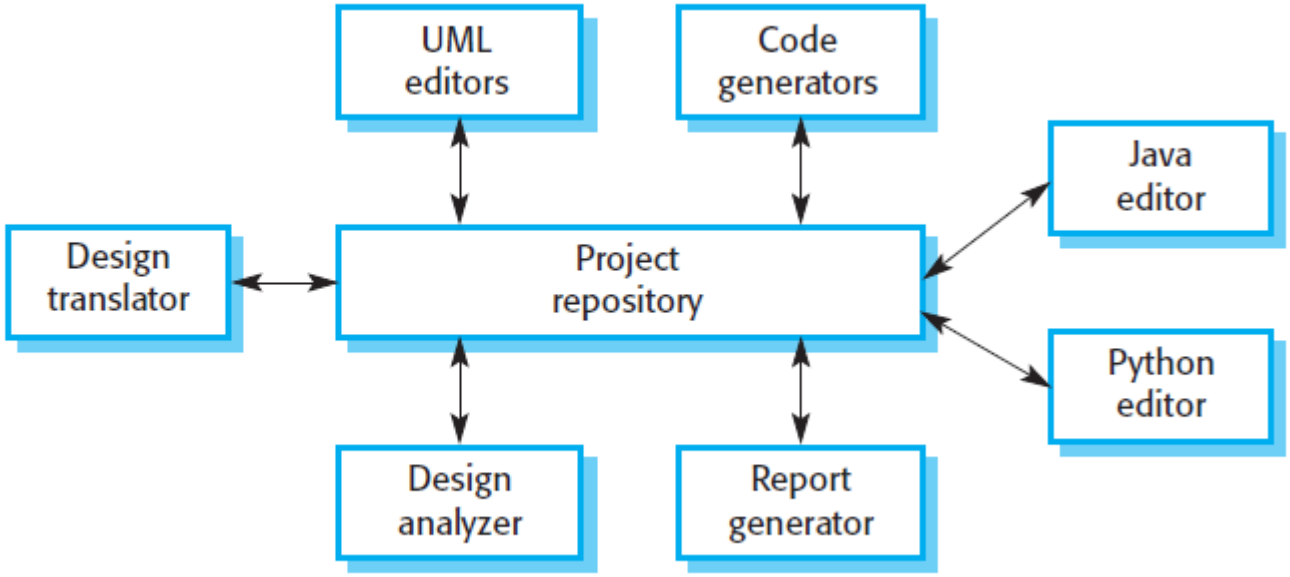


Figure 6.11 A repository architecture for an IDE

Client–Server Architectural Style

Name	Client–server
Description	In a client–server architecture, the system is presented as a set of services, with each service delivered by a separate server. Clients are users of these services and access servers to make use of them.
Example	Figure 6.13 is an example of a film and video/DVD library organized as a client–server system.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure and so is susceptible to denial-of-service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. Management problems may arise if servers are owned by different organizations.

Client–Server Architectural Style

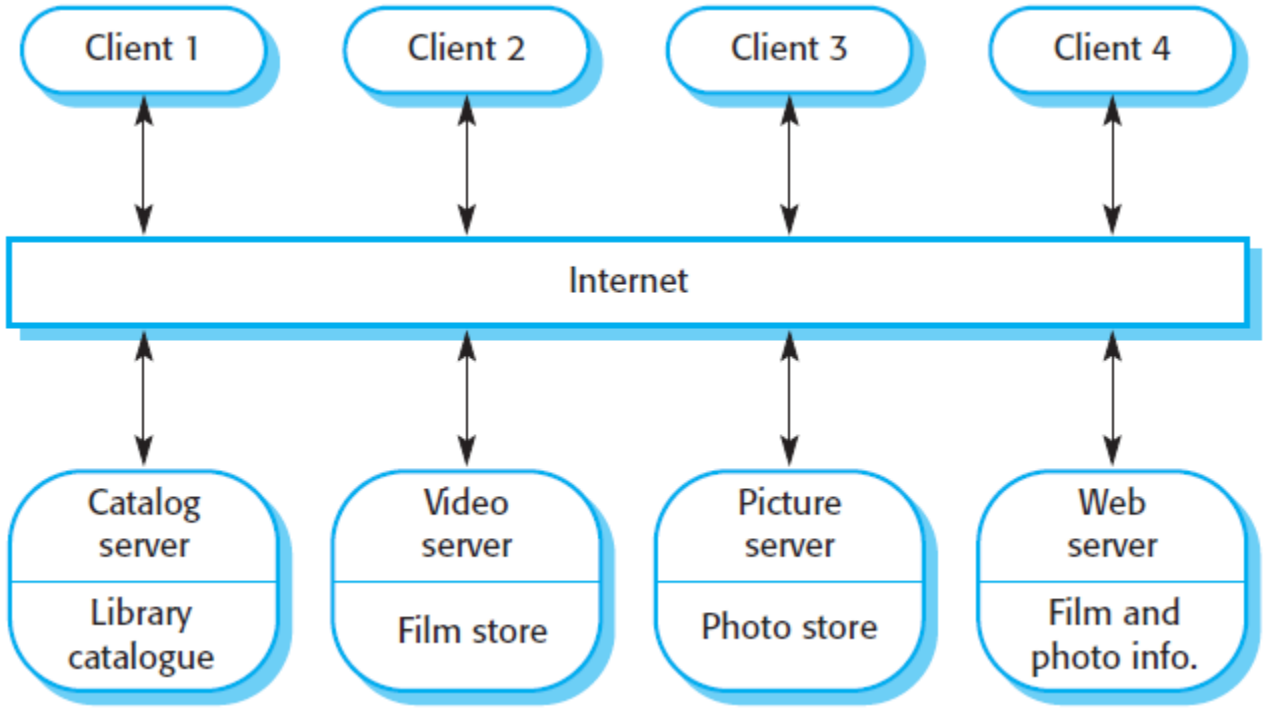
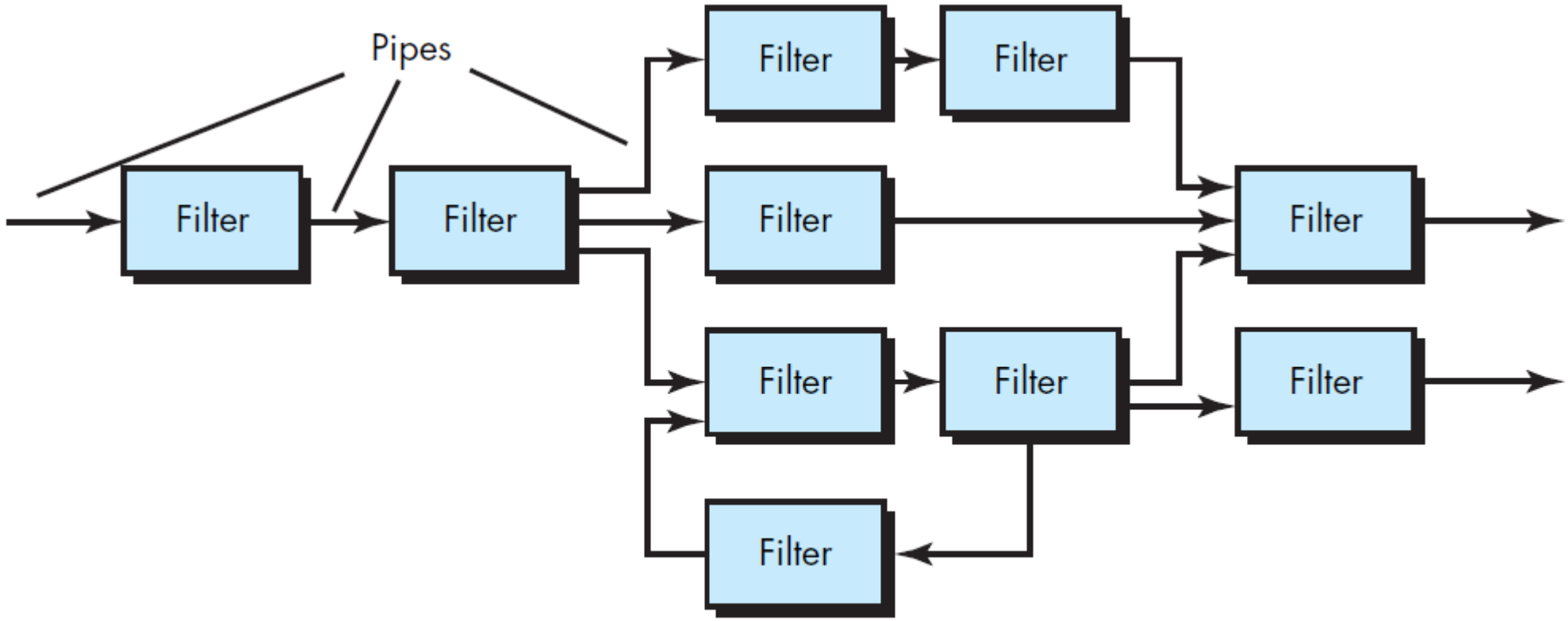


Figure 6.13 A client–server architecture for a film library

Pipe and Filter Architecture

Name	Pipe and filter
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
Example	Figure 6.15 is an example of a pipe and filter system used for processing invoices.
When used	Commonly used in data-processing applications (both batch and transaction-based) where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse architectural components that use incompatible data structures.

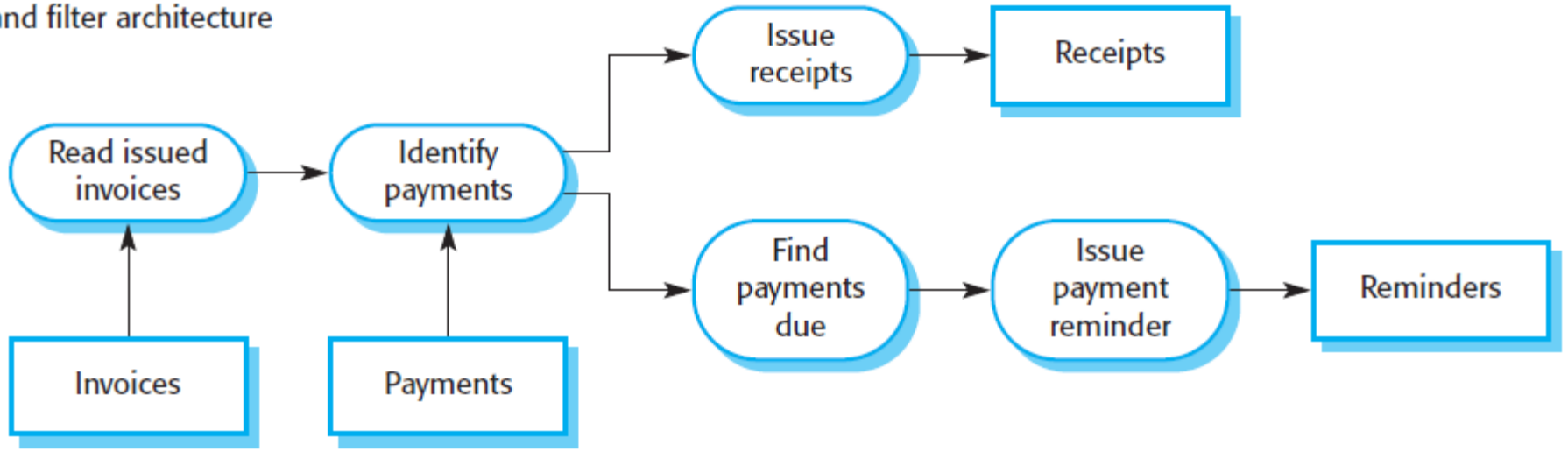
Pipe and Filter Architecture



Pipes and filters

Pipe and Filter Architecture

Figure 6.15 An example of the pipe and filter architecture



Architecture Description Language (ADL)

Architecture Description Language (ADL)

- Software architectures shift the focus of developers from lines-of-code to coarser-grained architectural elements and their overall interconnection structure.
- An ADL must explicitly model components, connectors, and their configurations; furthermore, to be truly usable and useful, it must provide tool support for architecture-based development and evolution.

Architecture Description Language (ADL)

- An ADL should exhibit the following properties:
  - ability to model components, with property assertions, interfaces, and implementations,
  - ability to model connectors, with protocols, property assertions, and implementations,
  - abstraction and encapsulation,
  - types and type checking, and
  - ability to accommodate analysis tools.

ADL Scope and Applicability

ADL	ACME	Aesop	C2	Darwin	MetaH	Rapide	SADL	UniCon	Weaves	Wright
Focus	Architectural interchange, predominantly at the structural level	Specification of architectures in specific styles	Architectures of highly-distributed, evolvable, and dynamic systems	Architectures of highly-distributed systems whose dynamism is guided by strict formal underpinnings	Architectures in the guidance, navigation, and control (GN&C) domain	Modeling and simulation of the dynamic behavior described by an architecture	Formal refinement of architectures across levels of detail	Glue code generation for interconnecting existing components using common interaction protocols	Data-flow architectures, characterized by high-volume of data and real-time requirements on its processing	Modeling and analysis (specifically, deadlock analysis) of the dynamic behavior of concurrent systems

Architecture Description Language (ADL)

- The focus on conceptual architecture and explicit treatment of connectors as first-class entities differentiate, ADLs from MILs (Module Interconnection Languages), programming languages, and OO notations and languages (e.g., UML)
- If certain abstractions (e.g., components and connectors) are buried in others (e.g., classes), the architect's job is made more (and unnecessarily) difficult; separating components from connectors, raising them both to visibility as top-level abstractions, and endowing them with certain features and limitations also raises them in the consciousness of the designer.

Architecture Description Language (ADL)

ADL Support for Modeling Components

ADL \ Features	Characteristics	Interface	Types	Semantics	Constraints	Evolution	Non-Functional Properties
ACME	*Component*; implementation independent	interface points are *ports*	extensible type system; parameterization enabled with templates	no support; can use other ADLs' semantic models in property lists	via interfaces only	structural subtyping via the *extends* feature	allows any attribute in property lists, but does not operate on them
Aesop	*Component*; implementation independent	interface points are *input* and *output ports*	extensible type system	(optional) style-specific languages for specifying semantics	via interfaces and semantics; stylistic invariants	behavior-preserving subtyping	allows association of arbitrary text with components
C2	*Component*; implementation independent	interface exported through top and bottom *ports*; interface elements are *provided* and *required*	extensible type system	component invariants and operation pre- and postconditions in 1st order logic	via interfaces and semantics; stylistic invariants	heterogeneous subtyping	none
Darwin	*Component*; implementation independent;	interface points are *services* (*provided* and *required*)	extensible type system; supports parameterization	π-calculus	via interfaces and semantics	none	none
MetaH	*Process*; implementation constraining	interface points are *ports*	Predefined, enumerated set of types	ControlH for modeling algorithms in the GN&C domain; implementation semantics via paths	via interfaces and semantics; modes; non-functional attributes	none	attributes needed for real-time schedulability, reliability, and security analysis
Rapide	*Interface*; implementation independent	interface points are *constituents* (*provides*, *requires*, *action*, and *service*)	extensible type system; contains a types sublanguage; supports parameterization	partially ordered event sets (posets)	via interfaces and semantics; algebraic constraints on component state; pattern constraints on event posets	inheritance (structural subtyping)	none
SADL	*Component*; implementation independent;	interface points are input and output *ports* (*iports* and *oports*)	extensible type system; allows parameterization of component signatures	none	via interfaces; stylistic invariants	subtyping by constraining supertypes; refinement via pattern maps	requires component modification (see Section 4.3.9)
UniCon	*Component*; implementation constraining	interface points are *players*	predefined, enumerated set of types	event traces in property lists	via interfaces and semantics; attributes; restrictions on players that can be provided by component types	none	attributes for schedulability analysis
Weaves	*Tool fragments*; implementation constraining	interface points are *read* and *write ports*; interface elements are *objects*	extensible type system; types are component *sockets*	partial ordering over input and output objects	via interface and semantics	none	allows association of arbitrary, uninterpreted annotations with components
Wright	*Component*; implementation independent;	interface points are *ports; port interaction semantics specified in CSP*	extensible type system; parameterizable number of ports and computation	not the focus; allowed in CSP	protocols of interaction for each port in CSP; stylistic invariants	via different parameter instantiations	none

Source: "A Classification and Comparison Framework for Software Architecture Description Languages"

Next On

Take Away Points

- The seven design principles
- Architectural design vs. Detailed design
- What software architecture is and its importance
- The architectural design decisions
- The architectural views
- The subtle difference between architectural pattern and style.
- The taxonomy of architectural styles
- What is an ADL?

Further Reading

- Otero, “Software Engineering Design”
  - Chapter 4: Patterns and Styles in Software Architecture
- Bass, Clements and Kazman, “Software Architecture in Practice”
  - Chapter 13: Architectural Tactics and Patterns
- Taylor, Medvidovic and Dashofy, “Software Architecture”
  - Chapter 3: Basic Concepts
  - Chapter 4: Designing Architectures
- Serebrenik et al., “Software Specification and Architecture”
  - https://www.win.tue.nl/~aserebre/2IW80/2014-2015/

Further Reading

- Ian Somerville, “Software Engineering”
  - Chapter 6: Architectural Design
- Pressman and Maxim, “Software Engineering”
  - Chapter 13: Architectural Design
- Medvidovic and Taylor, “A Classification and Comparison Framework for Software Architecture Description Languages”

Next Lecture

- This is the end of the module.